

Sky Analytics Engine: Architettura Distribuita per l'Analisi Big Data del Traffico Aereo

Sviluppo di una Pipeline di batch processing con Spark, NiFi e Airflow

Flavio Simonelli

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

flavio.simonelli@alumni.uniroma2.eu

Francesco Masci

Dipartimento di Ingegneria

Università di Roma "Tor Vergata"

francesco.masci.02@alumni.uniroma2.eu

Sommario—Il presente documento descrive in dettaglio la progettazione, lo sviluppo e la validazione sperimentale di Sky Analytics Engine, una piattaforma Big Data concepita per l'analisi distribuita e l'orchestrazione di pipeline di batch processing relative al traffico aereo statunitense. SAE integra componenti quali Apache Airflow per l'orchestrazione, Apache NiFi per l'ingestion asincrona e Apache Spark per il processamento distribuito su cluster Hadoop. L'architettura proposta si distingue per la sua estrema modularità, implementata attraverso un approccio di "polyglot persistence" e una netta separazione tra i piani di controllo e di esecuzione. Il sistema è in grado di gestire dataset massivi, spaziando dall'esecuzione di semplici query distribuite all'applicazione di tecniche avanzate di approssimazione algoritmica per il calcolo dei percentili, fino a modelli di machine learning non supervisionato per estrarre insight operativi. I risultati sperimentali confermano la superiorità delle API strutturate di Spark (DataFrame/SQL) rispetto all'approccio a basso livello (RDD) e dimostrano l'efficacia del formato colonnare Parquet nel ridurre drasticamente l'I/O overhead in ambienti distribuiti, sia in implementazioni on-premise che in scenari cloud-native.

Index Terms—Architetture Big Data, Apache Spark, Apache NiFi, Apache Airflow, Orchestrazione di Pipeline, Data Ingestion, Machine Learning, Scalabilità, Polyglot Persistence, Batch Processing.

I. INTRODUZIONE

La crescita esponenziale dei dati generati dai sistemi operanti nel settore del trasporto aereo richiede architetture di elaborazione che superino agilmente i limiti imposti dai sistemi monolitici tradizionali. La sfida odierna non riguarda esclusivamente il volume dei dati generati, ma coinvolge anche la velocità con cui questi devono essere acquisiti, trasformati e resi disponibili per le analisi, unita alla varietà dei formati sorgente e delle destinazioni di storage. Sky Analytics Engine è stato concepito e sviluppato per rispondere in modo esaustivo a queste sfide attraverso un'architettura distribuita, altamente modulare e "cloud-ready".

L'obiettivo primario di SAE è fornire una piattaforma che non si limiti ad essere un mero strumento di calcolo, bensì un ecosistema completo in cui ogni fase del ciclo di vita del dato (Ingestion, Storage, Processing, Serving e Monitoring) sia isolata, tracciabile e facilmente estensibile. Il caso d'uso affrontato in questo progetto si focalizza sull'analisi dei dati storici relativi ai ritardi dei voli commerciali negli Stati Uniti nel periodo Gennaio-Aprile 2025, forniti dal Bureau of

Transportation Statistics. Questo dominio applicativo offre una notevole complessità strutturale (come presenza di valori nulli, anomalie statistiche, dipendenze temporali) e rappresenta un banco di prova ideale per l'implementazione di una pipeline di batch processing in ambito Big Data.

Nelle sezioni successive, esploreremo in dettaglio come la modularità implementata a livello di codice sorgente, attraverso l'utilizzo estensivo del pattern GoF Factory e interfacce astratte in Java, si rifletta coerentemente nella modularità dell'infrastruttura di deployment. Questo design permette a SAE di operare con la medesima efficacia e senza alterazioni alla logica di business sia su un ambiente di sviluppo locale containerizzato tramite Docker Compose, sia su un cluster cloud di produzione basato su Amazon Web Services tramite i servizi di AWS EMR e/o EC2.

II. ARCHITETTURA DEL SISTEMA E MODULARITÀ

L'architettura di Sky Analytics Engine è stata concepita seguendo i rigidi principi della "Separation of Concerns" e della "Microservices-oriented Infrastructure". Il sistema rifugge l'approccio monolitico, configurandosi come un'aggregazione di servizi distribuiti coordinati che comunicano esclusivamente attraverso interfacce standardizzate.

A. Decomposizione Funzionale

L'infrastruttura SAE è organizzata in macro-livelli gerarchici indipendenti. Questa netta separazione assicura che il sistema sia "future-proof": è possibile scalare o sostituire il motore di processing o qualsiasi componente in generale senza impattare il sistema di orchestrazione o le logiche di visualizzazione.

I livelli principali sono:

- 1) **Orchestration Layer:** Gestisce lo scheduling e lo stato delle esecuzioni.
- 2) **Ingestion Layer:** Si occupa dell'acquisizione sicura e della trasformazione grezza dei dati.
- 3) **Processing Layer:** Esegue il calcolo distribuito e l'addestramento dei modelli ML.
- 4) **Storage Layer:** Implementa la persistenza multi-modello.
- 5) **Monitoring Layer:** Raccoglie la telemetria di sistema.

B. Livello di Orchestrazione

In Sky Analytics Engine, Apache Airflow 3.0 non viene utilizzato per un singolo flusso lineare, ma gestisce una suite di tre DAG specializzati, ciascuno progettato per coprire un aspetto specifico del ciclo di vita del software e della validazione sperimentale.

1) *Disaccoppiamento tramite Apache Livy*: Una caratteristica architetturale di rilievo è l'integrazione di Apache Livy. Nelle architetture legacy, Airflow necessita dell'installazione dei binari client di Spark e Hadoop sui propri nodi worker per lanciare i job tramite `spark-submit`. SAE supera questa limitazione interagendo esclusivamente con l'API REST di Livy. Airflow prepara un payload JSON contenente il path del JAR (precedentemente caricato su HDFS o S3) e gli argomenti di classe, inviandoli a Livy. Questo approccio trasforma di fatto il cluster Spark in una risorsa "Compute-as-a-Service", riducendo il carico computazionale sul nodo orchestratore e migliorando la sicurezza isolando il piano di controllo dal piano dati.

2) *Sincronizzazione Asincrona con NiFi*: L'interazione tra Airflow e NiFi rappresenta una sfida architetturale. SAE risolve questo problema con un pattern di callback asincrono. Airflow avvia il flusso NiFi tramite una chiamata HTTP POST, fornendo un JWT generato a runtime e un identificatore di variabile univoco. Successivamente, Airflow entra in uno stato di *Sensor Polling*, consumando risorse minime. Quando NiFi completa l'ingestion, utilizza il JWT per autenticarsi contro le API REST di Airflow e aggiornare la variabile di stato, sbloccando il DAG e permettendo l'avvio della fase di processing.

3) *DAG di Orchestrazione e Benchmarking*: L'orchestrazione dei flussi di lavoro in Sky Analytics Engine non si affida a un singolo processo lineare, ma è demandata a una suite di tre DAG specializzati su Apache Airflow. Ciascun workflow è ingegnerizzato per isolare specifiche fasi del ciclo di vita del dato o per automatizzare le sessioni di validazione sperimentale:

1) **Workflow Principale**: Coordina l'interazione end-to-end tra il livello di ingestion e quello di calcolo sfruttando il meccanismo di callback asincrono descritto in precedenza. Al fine di massimizzare la flessibilità operativa, il DAG supporta la parametrizzazione a runtime, abilitando tre diverse modalità di esecuzione:

- *Both*: Esegue la pipeline completa, dall'ingestion iniziale al processamento distribuito.
- *Ingest Only*: Arresta il flusso subito dopo la conversione dei record in Parquet e il caricamento nel Data Lake, utile per operazioni di puro popolamento.
- *Execution Only*: Salta la fase di ingestion, avviando direttamente le classi analitiche di Spark sui dati Parquet preesistenti, ottimizzando i tempi di sviluppo e debug.

2) **Benchmark Standalone**: Progettato specificamente per la raccolta sistematica dei dati di telemetria in ambienti locali o su istanze AWS EC2 standard. Al fine di garan-

tire l'assoluta affidabilità dei benchmark analitici, questo DAG genera dinamicamente le task dependency tramite un costrutto ciclico in Python, forzando un'esecuzione puramente sequenziale di tutte le combinazioni possibili tra le 4 query target e i 3 backend. Tale isolamento temporale impedisce la sovrapposizione computazionale e l'interferenza sull'I/O di rete o sul disco, restituendo metriche pulite.

3) **Benchmark Cloud-Native**: Sviluppato per automatizzare la valutazione sperimentale sull'infrastruttura fully-managed AWS EMR. Questo workflow bypassa il middleware Apache Livy e si interfaccia direttamente con i servizi infrastrutturali di Amazon Web Services sfruttando l'operatore *EmrAddStepsOperator* per iniettare i job Java sotto forma di step sequenziali nel Resource Manager YARN del cluster e delegando il tracciamento allo stato di *EmrStepSensor*. Questo design permette di sfruttare nativamente i meccanismi cloud di aggregazione dei log e la resilienza elastica dell'infrastruttura AWS.

C. Livello di Ingestion

Apache NiFi è la spina dorsale del livello di acquisizione dati. La sua architettura basata sul paradigma Flow-Based Programming garantisce un controllo visivo e transazionale sui flussi di dati, garantendo anche la Data Provenance.

1) *Pipeline di Trasformazione*: Il flusso `SAE_Ingestion_Pipeline` esegue un processo Extract-Load-Transform sofisticato:

- **Estrazione dei Metadati**: Il processore `EvaluateJsonPath` estrae dal payload di Airflow le configurazioni dinamiche (es. bucket S3 di destinazione, URL del dataset).
- **Fetch e Validazione**: `InvokeHTTP` scarica il dataset CSV zippato. Un processore custom calcola l'hash SHA-1 in streaming e lo confronta con la firma attesa, garantendo la non-ripudio del dato.
- **Conversione Colonnare Parquet**: Questo è il passaggio più critico per l'ottimizzazione delle performance a valle. Utilizzando `ConvertRecord` basato su uno schema Avro fortemente tipizzato, NiFi trasforma i dati da formato testuale CSV a formato colonnare Parquet. Questa operazione preserva i tipi nativi (Integer per i tempi, Double per i ritardi) e scarta i record malformati inviandoli a una coda di dead-letter.

2) *Formato dei Dati*: La scelta di Parquet è strategica: la sua struttura a blocchi con dizionari integrati abilita il *predicate pushdown* nel motore Catalyst di Spark. Se una query analizza solo American Airlines, Spark legge dai dischi HDFS/S3 esclusivamente i blocchi Parquet contenenti tale compagnia, riducendo l'I/O di rete di ordini di grandezza rispetto alla scansione lineare di un file CSV.

3) *Pre-processing e Data Quality*: La fase di ingestion non si limita alla mera conversione di formato, ma esegue un rigoroso controllo di qualità e pulizia del dato in streaming tramite processori NiFi dedicati. I dati grezzi, infatti,

presentano potenzialmente diverse inconsistenze, come valori di ritardo negativi sporchi, campi chiave mancanti e stati logici contraddittori.

La pipeline implementa due logiche fondamentali per garantire l'integrità analitica a valle:

- **Imputazione e Normalizzazione dei Ritardi:** Utilizzando uno script all'interno del processore `UpdateRecord`, la pipeline analizza i campi relativi alle cause di ritardo (*Carrier, Weather, NAS, Security, Late Aircraft*). Se esiste almeno una causa non nulla, le restanti cause nulle vengono imputate a 0.0, in quanto non hanno contribuito al ritardo. Contestualmente, eventuali valori negativi errati vengono corretti a 0.0. Tuttavia, se tutte le cause sono nulle, vengono mantenute tali. Questo riflette una precisa logica di business: per legge federale, le compagnie aeree non sono tenute a dichiarare i motivi specifici del ritardo se quest'ultimo è inferiore ai 15 minuti. Pertanto, i voli che registrano un ritardo all'arrivo effettivo (`ARR_DELAY > 0`) ma presentano componenti nulle, rappresentano ritardi lievi non classificabili. Questi record sono coerentemente ignorati nelle aggregazioni per causa (come richiesto dalla Query 2), preservando la fedeltà del dato.
- **Filtraggio delle Inconsistenze Logiche:** Attraverso l'uso di query SQL sul processore `QueryRecord`, vengono scartati i record anomali (instradati su una coda di *dead-letter*) che presentano discrepanze non recuperabili, quali: assenza di campi temporali o identificativi della compagnia, voli contrassegnati come cancellati ma recanti un orario di partenza effettivo, voli non cancellati ma privi di orario di arrivo/partenza, oppure evidenti errori matematici (es. ritardo totale negativo a fronte di una somma delle cause di ritardo positiva).

D. Livello di Processing

Il cuore computazionale di SAE è l'applicazione Spark, sviluppata in Java 11. La modularità del codice sorgente è stata ottenuta applicando rigorosamente i design pattern della programmazione orientata agli oggetti.

1) *Pattern Factory e Repository:* Per astrarre l'accesso ai dati, SAE implementa il pattern Repository. L'interfaccia `FlightRepository` definisce metodi generici come `getFlights()` e `saveResults()`. Tramite la classe `FlightRepositoryFactory`, l'applicazione istanzia a runtime implementazioni specifiche (es. `HdfsFlightRepository`, `PostgresFlightRepository`, `RedisFlightRepository`) basandosi su un file di configurazione YAML, gestiti attraverso la classe `ApplicationConfig.java`. Questo significa che la logica di business analitica è completamente disaccoppiata dalla tecnologia di storage sottostante.

2) *Astrazione delle Query:* Ogni query estende la classe astratta `BaseQuery`. Questa classe si fa carico del boilerplate code: istanziazione della `SparkSession`, gestione del logging strutturato, iniezione del listener delle metriche

e recupero dei dati dal repository. Le classi figlie devono implementare esclusivamente tre metodi: `runQueryRDD`, `runQueryDataFrame` e `runQuerySQL`, focalizzandosi solo sulla trasformazione del dato.

E. Livello di Storage

Un'architettura Big Data moderna non può fare affidamento su un singolo paradigma di database. SAE implementa una strategia "Polyglot Persistence" per ottimizzare i costi e le latenze di accesso in base al caso d'uso:

- **Hadoop HDFS / Amazon S3:** Fungono da *Single Source of Truth*. Conservano i dati raw e pre-processati in Parquet, offrendo throughput elevato per le letture batch di Spark.
- **PostgreSQL:** Utilizzato come *Serving Layer* relazionale. I risultati aggregati delle query vengono salvati qui per essere facilmente interrogati da strumenti di Business Intelligence tradizionali come Grafana.
- **Redis Stack:** Funge da database per le metriche di telemetria emesse in tempo reale da Spark.
- **Apache HBase:** Implementa il paradigma NoSQL a colonne larghe. È impiegato per memorizzare dataset di risultati estremamente granulari, garantendo tempi di accesso nell'ordine dei millisecondi per singole chiavi.

III. DETTAGLI IMPLEMENTATIVI DELLA COMPUTAZIONE

Le quattro query richieste dal progetto sono state implementate in modo da testare diverse astrazioni di Spark e valutare i trade-off tra controllo a basso livello ed engine ottimizzati.

A. Analisi Base e Ottimizzazioni RDD

La Query 1 richiede il calcolo di statistiche mensili (media, min, max, tasso di cancellazione) per specifiche compagnie. Nell'implementazione RDD, si è evitato l'uso dell'operatore `groupByKey`, noto per causare gravi problemi di Out-Of-Memory a causa del trasferimento incontrollato di dati durante la fase di shuffle. Al suo posto è stato utilizzato `aggregateByKey`. Questo operatore combina i valori parzialmente all'interno di ogni singola partizione del nodo worker prima di trasferirli sulla rete, riducendo l'impronta di memoria e i tempi di rete.

La Query 2 introduce logiche di *feature engineering* per gestire i dati sparsi, imputando i valori nulli delle cause di ritardo a zero e filtrando i carrier non statisticamente significativi.

B. Approssimazione tramite KLL Sketch

Il calcolo dei percentili su finestre orarie nella Query 3 rappresenta la sfida computazionale più ardua. Un calcolo esatto richiederebbe l'ordinamento globale di milioni di record, un'operazione che su un cluster distribuito causa enormi colli di bottiglia dovuti allo scambio di dati.

SAE risolve questo problema implementando tecniche di "Quantile Sketching". Attraverso l'integrazione della libreria Apache DataSketches, è stato implementato lo stimatore Karnin-Lang-Liberty. L'algoritmo KLL mantiene in memoria

un set compresso e rappresentativo dei dati osservati. La potenza matematica del KLL risiede nella sua proprietà di unione: due sketch generati su nodi worker differenti possono essere sommati nel driver node mantenendo rigorose garanzie matematiche sull'errore di rango massimo (configurato di default a $\sim 1.6\%$). Questo permette di calcolare i percentili orari in un singolo passaggio sui dati, con una complessità spaziale fissa indipendente dalla dimensione del dataset.

Nell'implementazione `DataFrame`, SAE delega questa approssimazione alla funzione nativa `percentile_approx`, la quale utilizza un approccio simile derivato dall'algoritmo Greenwald-Khanna.

C. Machine Learning e Ottimizzazione Modelli

L'obiettivo della *Query 4* è raggruppare le compagnie aeree in base alle loro feature prestazionali.

L'implementazione in `AirlineClustering.java` costruisce una complessa pipeline:

- 1) **Aggregazione Iniziale:** Calcolo delle feature medie per carrier escludendo i voli cancellati dai conteggi di ritardo.
- 2) **Vector Assembler:** Consolidamento delle colonne numeriche in una singola colonna di tipo `Vector`, formato richiesto da `MLlib`.
- 3) **Standardization:** Applicazione di uno `StandardScaler`. Poiché il k-Means è sensibile alla scala delle feature (basandosi sulla distanza euclidea), è imperativo normalizzare i dati affinché abbiano media 0 e deviazione standard 1, evitando che feature con magnitudo maggiore dominino l'algoritmo.
- 4) **Model Selection:** SAE non richiede di specificare a priori il parametro K , ovvero il numero di cluster. Il sistema esegue un loop iterativo addestrando modelli da $K = 2$ a $K = 8$. Per ogni modello, valuta la qualità del raggruppamento utilizzando il *Silhouette Score* fornito da `ClusteringEvaluator`. Il punteggio di *Silhouette* misura la coesione intra-cluster rispetto alla separazione inter-cluster. SAE seleziona dinamicamente il K che massimizza questo punteggio, garantendo un clustering statisticamente fondato.

IV. TELEMETRIA DISTRIBUITA

Per garantire l'osservabilità di un sistema distribuito asincrono, SAE implementa un modulo di telemetria altamente personalizzato.

A. JobTimerListener Custom

Affidarsi unicamente alla UI nativa di Spark non è sufficiente per analisi storiche automatizzate. SAE implementa la classe `JobTimerListener`, che estende le interfacce native dello `SparkListener`. Questo componente viene agganciato al bus degli eventi di Spark durante l'inizializzazione della sessione.

Il Listener intercetta eventi cruciali come `onJobStart`, `onStageCompleted` e `onTaskEnd`. Registra timestamp millisecondi e metriche relative al volume di dati letti e scritti.

Alla fine del ciclo di vita dell'applicazione, aggrega questi dati e, sfruttando la connettività asincrona verso Redis, persiste i record di telemetria. Questo permette di scorporare il "vero" tempo di calcolo sui dati dal tempo fisiologico speso da Spark per negoziare le risorse con YARN o allocare i container.

B. Visualizzazione in Grafana

L'ecosistema si chiude con Grafana, configurato per leggere direttamente da Redis (tramite plugin `RedisSearch`) e PostgreSQL. Le dashboard sono state progettate per presentare:

- Benchmark prestazionali comparativi.
- Trend temporali dei ritardi suddivisi per vettore.
- Heatmap della distribuzione oraria calcolata sui percentili approssimati.
- La disposizione geometrica dei cluster ML nello spazio delle feature principali.

V. INFRASTRUTTURA DI DEPLOYMENT

L'agilità di SAE è esaltata dalle sue procedure di deployment, strutturate in due macro-scenari.

A. Configuration Management as Code

Tutta la configurazione è externalizzata in file `YAML` e `.env`. L'assenza di stringhe di connessione o path assoluti hardcoded nel codice Java permette di compilare il JAR una sola volta (tramite Maven) e di eseguirlo in contesti completamente diversi.

B. Ambiente Standalone AWS EC2

Il primo macro-scenario simula un'infrastruttura di tipo locale o on-premise virtualizzata in cloud. Il deployment si avvale di una suite di istanze Amazon EC2 attestate all'interno della medesima Virtual Private Cloud e collegate in una sottorete privata:

- **Master Node (1x t3.medium):** Ospita i servizi di coordinamento primari, tra cui Apache Airflow, Apache NiFi, Apache Livy, il master di Apache Spark, e i database di serving PostgreSQL e Redis. Dispone di 2 vCPU e 4 GB di RAM.
- **Worker Nodes (3x t3.small):** Dedicati esclusivamente al calcolo distribuito.
- Ciascun nodo ospita un demone worker di Spark e un datanode HDFS all'interno di container Docker coordinati tramite Docker Compose. Ciascun worker dispone di 2 vCPU e 2 GB di RAM.

In questa configurazione, HDFS funge da file system distribuito primario, mentre la sottomissione dei job Spark avviene in modalità client tramite Apache Livy, garantendo il completo disaccoppiamento logico ed architetturale.

C. Ambiente Cloud-Native AWS EMR

Il secondo scenario proietta SAE in una dimensione pienamente elastica ed enterprise-ready, sfruttando il servizio gestito Amazon Elastic MapReduce. L'infrastruttura è configurata come segue:

- **Master Node (1x m6g.xlarge):** Coordina il resource manager Hadoop YARN.
- **Core Nodes (3x m6g.xlarge):** Eseguono i demoni executor di Spark sotto il controllo di YARN e memorizzano i dati intermedi. Ciascun nodo dispone di 4 vCPU e 16 GB di RAM.

Il vantaggio cardine di questo scenario risiede nell'integrazione nativa dello storage con Amazon S3 mediante il protocollo `s3a://`. Questo abilita un paradigma **Shared-Nothing** puro, in cui le risorse di calcolo possono essere scalate o spente in modo del tutto indipendente dallo storage persistente, azzerando i costi di idle e ottimizzando il total cost of ownership. L'orchestrazione da Airflow avviene in questo caso sfruttando le API native di AWS tramite operatori Spark dedicati.

VI. VALUTAZIONE SPERIMENTALE

Definita l'architettura e implementati i componenti, è stata condotta una fase di validazione sperimentale sistematica per valutare la robustezza, la scalabilità e le performance di Sky Analytics Engine. I test sono stati orchestrati in modo sequenziale tramite il DAG `ec2_benchmark` su EC2 e `emr_benchmark` su EMR, garantendo l'assenza di interferenze computazionali e l'affidabilità della telemetria raccolta.

A. Analisi Comparativa delle Performance

Le misurazioni prestazionali raccolte evidenziano trend marcati circa l'efficacia dei diversi backend di Apache Spark e l'impatto dell'infrastruttura di deployment.

1) *Dispersione Statistica dei Backend:* L'osservazione dei box plot (Fig. 1) rivela che le API strutturate presentano mediane prestazionali generalmente inferiori rispetto all'approccio RDD nelle query più complesse (Query 2 e Query 3), mostrando anche una dispersione statistica più contratta nella maggior parte dei casi. La scatola del box plot per DataFrame e SQL risulta estremamente stretta, con code di variabilità minime e una quasi totale assenza di outlier significativi nel caso della Query 1 e 3 su EMR.

Questa notevole stabilità prestazionale è una conseguenza diretta del **Catalyst Optimizer** e del motore d'esecuzione **Tungsten**. Catalyst esegue l'ottimizzazione logica del piano di esecuzione (attraverso regole come il predicate pushdown e la column projection) prima di compilare il bytecode Java, assicurando che la pipeline fisica sia estremamente lineare ed efficiente. Dal canto suo, Tungsten memorizza i record in formato binario off-heap compresso, aggirando del tutto l'allocazione di oggetti Java sulla JVM. Ciò riduce a livelli trascurabili l'attività del Garbage Collector, neutralizzando una delle fonti principali di variabilità temporale nei sistemi distribuiti.

Al contrario, le esecuzioni basate su RDD mostrano box più alti e ampi, indicando una marcata dispersione statistica e code di esecuzione lunghe. Non avendo alcuna visibilità sulla struttura dei dati, Spark tratta i record RDD come oggetti Java opachi, costringendo ad operazioni di serializzazione

e deserializzazione pesante per ogni operazione di trasformazione o filtraggio. Questo continuo passaggio di oggetti generici carica pesantemente la memoria degli heap della JVM, innescando frequenti e imprevedibili cicli di garbage collection che dilatano casualmente i tempi di esecuzione, producendo la dispersione visibile in Fig. 1.

Inoltre, nella Query 3, la dispersione del backend RDD è accentuata dall'overhead algoritmico del KLL Sketch implementato tramite la libreria esterna Apache DataSketches integrata manualmente. I backend DataFrame e SQL beneficiano invece dell'ottimizzazione nativa della funzione `percentile_approx` basata sull'algoritmo di Greenwald-Khanna incorporato direttamente nelle routine Tungsten, che mantiene un footprint in memoria costante e una dispersione minima.

2) *Impatto dell'Infrastruttura:* Un aspetto cruciale che emerge dall'analisi comparativa è l'impatto dell'infrastruttura sul cosiddetto **Wall Clock Time** (il tempo totale reale percepito dall'utente per completare il job). Come evidenziato dai grafici dettagliati posizionati in Appendice A, esiste un overhead di Wall Clock Time che si somma al tempo puro di calcolo degli executor Spark.

Questo overhead della telemetria è causato da fattori estranei al processamento dei dati in senso stretto:

- **Inizializzazione e allocazione delle risorse:** Il tempo speso dal Driver per comunicare con il Resource Manager per allocare i container degli executor.
- **Compilazione del DAG:** La pianificazione logica e fisica eseguita da Catalyst a runtime.
- **Negoziiazione della rete ed I/O:** La serializzazione e la spedizione dei task compilati dal driver ai vari nodi di calcolo.

Questo tempo morto incide proporzionalmente di più sulle query rapide (Query 1 e 2). L'infrastruttura gioca un ruolo determinante nel mitigare questo overhead.

B. Discussione dei Risultati Analitici

1) *Query 1: Performance Mensili delle Compagnie:* I risultati analitici estratti per American Airlines e Delta Air Lines rivelano comportamenti operativi divergenti durante il quadrimestre Gennaio-Aprile 2025 (Fig. ??). In particolare, a Gennaio si riscontra il picco massimo nel tasso di cancellazione per entrambe le compagnie (AA: 3,65%, DL: 2,67%).

Tuttavia, l'andamento del ritardo medio di partenza evidenzia una differenza sostanziale nell'efficienza operativa post-invernale:

- **Delta Air Lines:** Mostra una costante e progressiva riduzione del ritardo medio di partenza, scendendo da 12,82 minuti a Gennaio fino a un eccellente 8,71 minuti ad Aprile, accompagnato da una drastica riduzione del tasso di cancellazione che si stabilizza sotto lo 0,50% (con un minimo dello 0,21% a Febbraio).
- **American Airlines:** Al contrario, AA sperimenta un costante incremento del ritardo medio di partenza, che sale da 12,89 minuti a Gennaio a ben 15,96 minuti ad



Figura 1. Dispersione statistica dei tempi di esecuzione su AWS EC2 (sinistra) e AWS EMR (destra) per la Query 1 (in alto), Query 2 (al centro) e Query 3 (in basso). I grafici evidenziano la distribuzione dei tempi per ciascun backend.

Aprile, mantenendo un tasso di cancellazione fisso attorno all'1, 40%.

2) Query 2: Decomposizione delle Cause di Ritardo:

La classifica dei top 10 vettori per ritardo medio all'arrivo (Fig. ??) rivela dinamiche profonde inerenti alla struttura delle compagnie aeree statunitensi. Il vettore in assoluto più inefficiente risulta essere **PSA Airlines (OH)**, con un ritardo medio all'arrivo di ben 17,16 minuti. L'analisi disaggregata delle cause mostra che OH soffre del valore più alto in assoluto di *late aircraft delay* (ritardo dovuto all'arrivo tardivo del velivolo precedente), pari a ben 43,51 minuti di media condizionata. Questo dato evidenzia il classico fenomeno di **propagazione dei ritardi**, dovuto probabilmente al fatto che OH pianifica voli estremamente serrati con tempi di turnaround minimi; di conseguenza, un singolo ritardo accumulato al mattino si propaga geometricamente su tutti i voli successivi della giornata.

All'estremo opposto, **United Airlines (UA)** mostra il ritardo all'arrivo medio più basso tra i grandi vettori tradizionali (2,46 minuti), ma registra il valore più elevato di *NAS delay* (ritardi imputabili al sistema di controllo del traffico aereo nazionale), pari a 21,33 minuti. Questo ci evidenzia che UA, pur gestendo in modo efficiente le proprie operazioni interne (basso carrier delay di 18,85 minuti), i suoi voli vengono costantemente penalizzati dalla congestione dello spazio aereo.

3) Query 3: Distribuzione Oraria dei Percentili di Ritardo:

L'analisi oraria dei percentili di ritardo (riportata in Appendice ??) rivela marcate discrepanze strutturali tra i dati analizzati, riflettendo le rispettive efficienze operative e la

resilienza del network di ciascuna compagnia.

Per favorire un confronto immediato, viene discusso in dettaglio il confronto diretto tra American Airlines e Delta Air Lines, le due compagnie che mostrano i trend più divergenti e rappresentativi. Nelle prime ore del mattino (dalle 05:00 alle 09:00), i ritardi sono minimi per entrambe le compagnie, con il 50° percentile e persino il 75° percentile stabili vicino allo zero o a valori negativi (indicando arrivi in anticipo). Ciò è dovuto al fatto che i primi voli della giornata utilizzano aeromobili che hanno sostato negli aeroporti durante la notte, azzerando la propagazione del giorno precedente.

Tuttavia, a partire dalle ore 12:00, si assiste a una crescita costante di tutti i percentili, che raggiunge il picco massimo nella fascia serale (dalle 18:00 alle 22:00). In questa finestra emerge la discrepanza fondamentale:

- **American Airlines:** Mostra una severa incapacità di assorbire i ritardi accumulati durante la giornata. Il 90° percentile di AA sale rapidamente nel pomeriggio superando la soglia critica dei 45 minuti e mantenendosi elevatissimo fino a tarda notte. Questo denota una propagazione non controllata dovuta a tempi di sosta troppo ristretti e congestione sistematica nei propri hub.
- **Delta Air Lines:** Evidenzia invece un controllo notevole dell'effetto accumulazione: pur registrando una crescita serale, i percentili di DL rimangono significativamente più bassi e mostrano una pendenza di crescita assai più dolce, segno di una pianificazione con margini di recupero più robusti ed efficienti routine di turnaround negli hub.

4) *Query 4: Clustering delle Compagnie Aeree*: L'applicazione dell'algoritmo K-Means con selezione automatica del modello tramite Silhouette Score ha identificato $K = 4$ come il numero ottimale di raggruppamenti statistici per descrivere i vettori (Fig. ??). La proiezione spaziale tramite **Principal Component Analysis** (Fig. ??) e la matrice di classificazione delineano quattro cluster operativi ben definiti (i valori medi delle feature per ciascun cluster sono riportati in Fig. ?? e sono stati normalizzati per evidenziare le differenze relative tra i cluster, mentre la Tabella ?? riporta i corrispondenti valori operativi reali per ogni vettore):

- **Cluster 0 - Alta Efficienza e Operazioni Scalabili**: Include UA, NK, B6, DL, MQ, YX, AS, WN. Questo è il cluster più numeroso ed è caratterizzato da ritardi medi molto contenuti.
- **Cluster 1 - Alta Inefficienza e Volatilità Operativa**: Comprende AA, F9, OH. Questi vettori mostrano ritardi medi elevati, ma soprattutto un'altissima variabilità (deviazione standard superiore a 70 minuti, con OH che raggiunge 82,75 minuti) e tassi di cancellazione elevati (OH detiene il record negativo del 6,09%). Questo cluster raggruppa vettori afflitti da gravi colli di bottiglia operativi, in cui l'instabilità delle connessioni e la propagazione dei ritardi rendono i tempi di volo altamente imprevedibili per l'utente finale.
- **Cluster 2 - Sensibilità Meteorologica ed Operativa**: Formato da OO e G4. Come discusso nella Query 2, questi vettori presentano una forte dipendenza da fattori esterni: registrano ritardi meteorologici estremi (12,11 minuti per OO, 8,92 minuti per G4) uniti ad alti carrier delay, collocandosi in una posizione intermedia dello spazio PCA, caratterizzata da un profilo di rischio meteorologico non trascurabile.
- **Cluster 3 - Isolamento Geografico**: Costituito esclusivamente da HA. HA si mappa come un *outlier* spaziale netto nella proiezione PCA. Presenta una combinazione unica al mondo: NAS delay quasi a zero (0,58 minuti) e un tasso di deviazione minimo, dovuto al fatto che il suo network inter-insulare opera al di fuori dei corridoi congestionati del continente, rendendo le sue dinamiche del tutto non confrontabili con qualsiasi altra compagnia statunitense.

VII. CONCLUSIONI E SVILUPPI FUTURI

Il progetto Sky Analytics Engine dimostra che l'implementazione rigorosa di pattern architetturali distribuiti e l'adozione di formati dati ottimizzati sono requisiti imprescindibili per gestire la complessità inerente ai Big Data. L'orchestrazione asincrona tra Airflow e NiFi, combinata con la modularità polyglot dell'engine Spark, fornisce un ambiente robusto ed enterprise-ready. L'introduzione di tecniche algoritmiche avanzate, quali i Quantile Sketches e il Model Selection automatizzato tramite Silhouette Score, dota SAE di capacità analitiche di altissimo livello pur mantenendo confinati i costi computazionali.

I potenziali sviluppi futuri della piattaforma includono la transizione da un paradigma puramente batch a un'architettura ibrida Lambda. L'integrazione di Apache Kafka e Spark Structured Streaming permetterebbe a SAE di acquisire eventi di volo in real-time, applicando i modelli ML di clustering precedentemente addestrati per prevedere ritardi anomali "on-the-fly".

RIFERIMENTI BIBLIOGRAFICI

- [1] Apache Spark Foundation, "Apache Spark - A Unified engine for large-scale data analytics," <https://archive.apache.org/dist/spark/docs/3.5.1/>, Consultato: Maggio 2026.
- [2] Apache Livy, "Apache Livy Documentation," <https://livy.apache.org/docs/latest/>, Consultato: Maggio 2026.
- [3] Apache Airflow, "TaskFlow API Tutorial," https://airflow.apache.org/docs/apache-airflow/stable/tutorial_taskflow_api.html, Consultato: Maggio 2026.
- [4] Apache NiFi, "Apache NiFi User Guide," <https://nifi.apache.org/docs/nifi-docs/html/user-guide.html>, Consultato: Maggio 2026.
- [5] Apache Hadoop, "HDFS Architecture Guide," <https://hadoop.apache.org/docs/r3.3.2/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>, Consultato: Maggio 2026.
- [6] Amazon Web Services, "Amazon Simple Storage Service (S3) Documentation," <https://docs.aws.amazon.com/s3/>, Consultato: Maggio 2026.
- [7] Apache Parquet, "Parquet Format Specification," <https://parquet.apache.org/docs/file-format/>, Consultato: Maggio 2026.
- [8] Apache HBase, "Apache HBase Reference Guide," <https://hbase.apache.org/book.html>, Consultato: Maggio 2026.
- [9] PostgreSQL Global Development Group, "PostgreSQL Documentation," <https://www.postgresql.org/docs/15/>, Consultato: Maggio 2026.
- [10] Redis Ltd., "Redis Documentation," <https://redis.io/docs/latest/>, Consultato: Maggio 2026.
- [11] Grafana Labs, "Grafana Official Documentation," <https://grafana.com/docs/>, Consultato: Maggio 2026.



Figura 2. Valori medi delle feature per ciascun cluster identificato nella Query 4. Le barre rappresentano le medie normalizzate delle singole features per permettere un confronto relativo tra i cluster.

APPENDICE A

DETTAGLIO DEI TEMPI DI ESECUZIONE

In questa appendice vengono riportati in dettaglio i pannelli di telemetria estratti dalle dashboard di Grafana, relativi ai tempi di esecuzione delle quattro query di processamento Spark.

Per ogni query viene presentato prima il pannello relativo all'ambiente **AWS EC2 Standalone** e, immediatamente a seguire, il pannello relativo all'ambiente **AWS EMR**.

Ogni pannello all'interno della plancia di monitoraggio è posizionato in modo da fornire una visione granulare delle performance del sistema, organizzato su tre livelli logici:

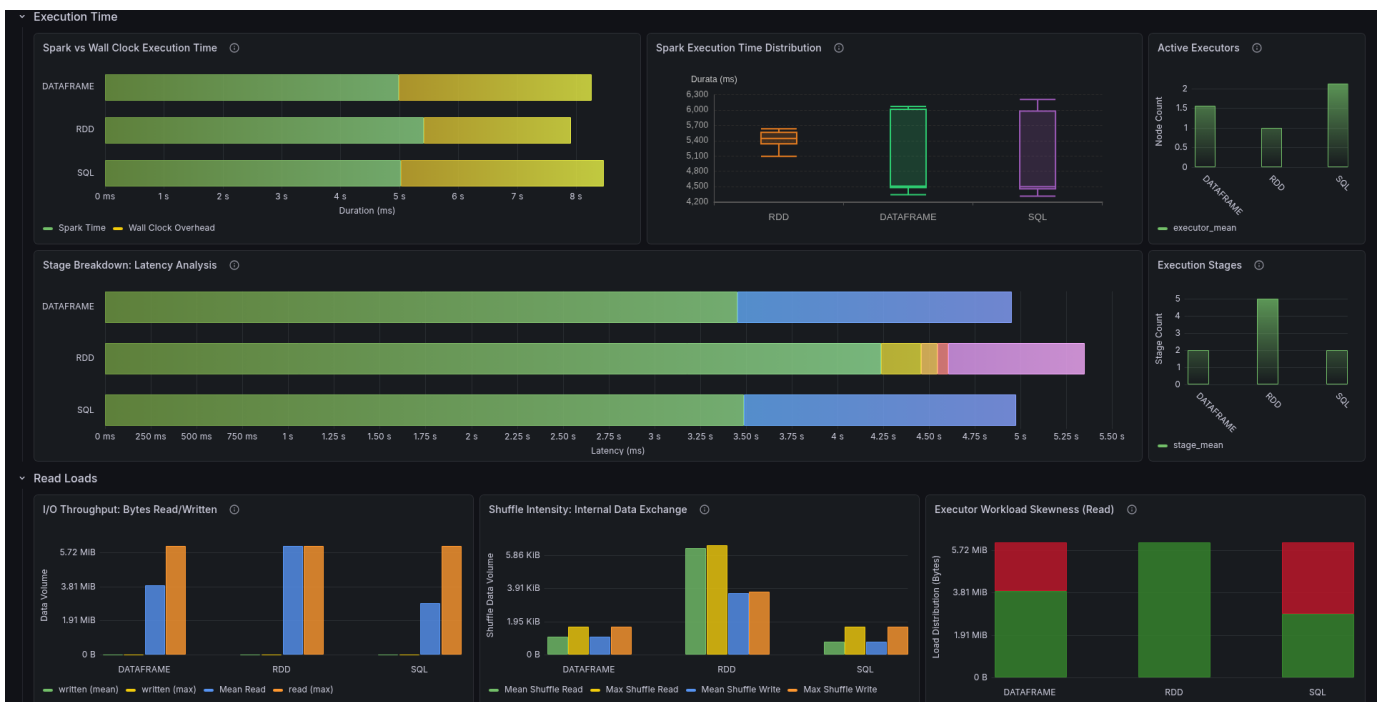
- **Spark vs Wall Clock Execution Time (In alto a sinistra):** Un grafico che confronta l'overhead misurato tramite `System.currentTimeMillis()` sul driver Java con il tempo di esecuzione effettivo dei job intercettato dai listener di Spark. Il divario evidenzia il tempo speso per lo scheduling e la negoziazione delle risorse.
- **Distribuzione Statistica dello Spark Time (In alto al centro):** Mostra la dispersione statistica dei tempi di calcolo di Spark attraverso le diverse run, permettendo di identificare la stabilità delle performance per il backend selezionato.
- **Active Executors (In alto a destra):** Indica il numero medio di nodi executor che hanno partecipato attivamente alla computazione durante le varie esecuzioni del benchmark.
- **Stage Breakdown (Al centro a sinistra):** Uno spaccato temporale dettagliato che scompone l'esecuzione nei singoli stage di computazione di Spark, utile per isolare la fase logica della query che presenta la latenza maggiore.
- **Execution Stages (Al centro a destra):** Rappresenta il numero medio di stage generati dall'ottimizzatore di Spark durante le esecuzioni, fornendo un'indicazione della complessità del piano fisico.
- **I/O Throughput (In basso a sinistra):** Monitoraggio del volume di dati letti e scritti su disco/HDFS/S3, che permette di valutare l'impatto delle operazioni di input/output persistente.
- **Shuffle I/O (In basso al centro):** Analisi del traffico dati generato esclusivamente durante le fasi di shuffle per la redistribuzione dei record tra i nodi del cluster.
- **Executor Workload Skew (In basso a destra):** Valuta il bilanciamento del carico tra i nodi. Il grafico confronta il valore medio dei byte letti (caso ideale di distribuzione uniforme, in verde) con il valore massimo letto da un singolo nodo (in rosso), evidenziando lo sbilanciamento del carico di lavoro rispetto alla configurazione ideale.



Figura 3. Telemetria Query 1 - Ambiente AWS EC2 Standalone.



Figura 4. Telemetria Query 1 - Ambiente AWS EMR.



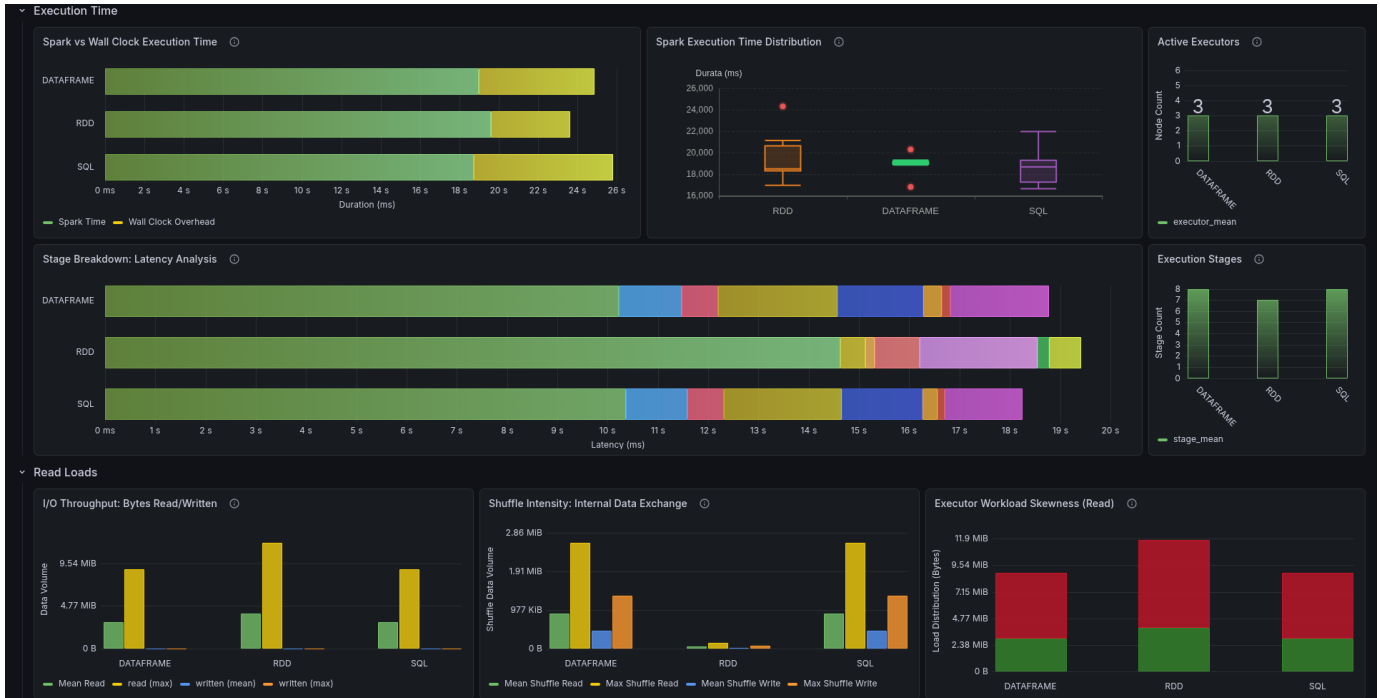


Figura 7. Telemetria Query 3 - Dettaglio Tempi AWS EC2 Standalone.



Figura 8. Telemetria Query 3 - Dettaglio Tempi AWS EMR.



Figura 9. Telemetria Query 4 - Ambiente AWS EC2 Standalone.



Figura 10. Telemetria Query 4 - Ambiente AWS EMR.

APPENDICE B
RISULTATI ANALITICI DELLE QUERY

In questa appendice vengono riportati in dettaglio i risultati analitici ottenuti dall'applicazione della pipeline SAE.

A. Query 1

Per la Query 1, vengono presentati i risultati relativi al trend dei ritardi medi mensili e al tasso di cancellazione per le compagnie aeree in esame.



Figura 11. Risultati Query 1: Trend dei ritardi medi mensili.

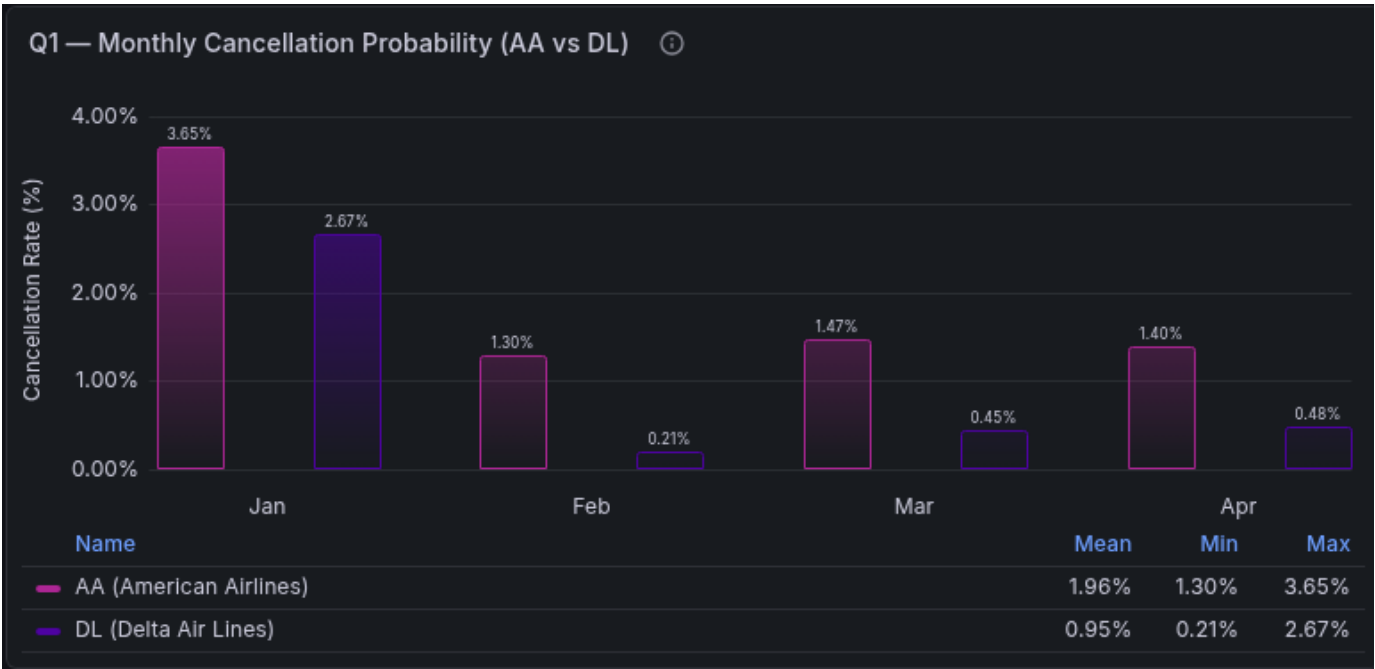


Figura 12. Risultati Query 1: Tasso e probabilità di cancellazione.

B. Query 2

Nella Query 2, vengono presentati i risultati relativi alla classifica delle tratte con i ritardi medi più elevati e la composizione percentuale delle cause di ritardo.

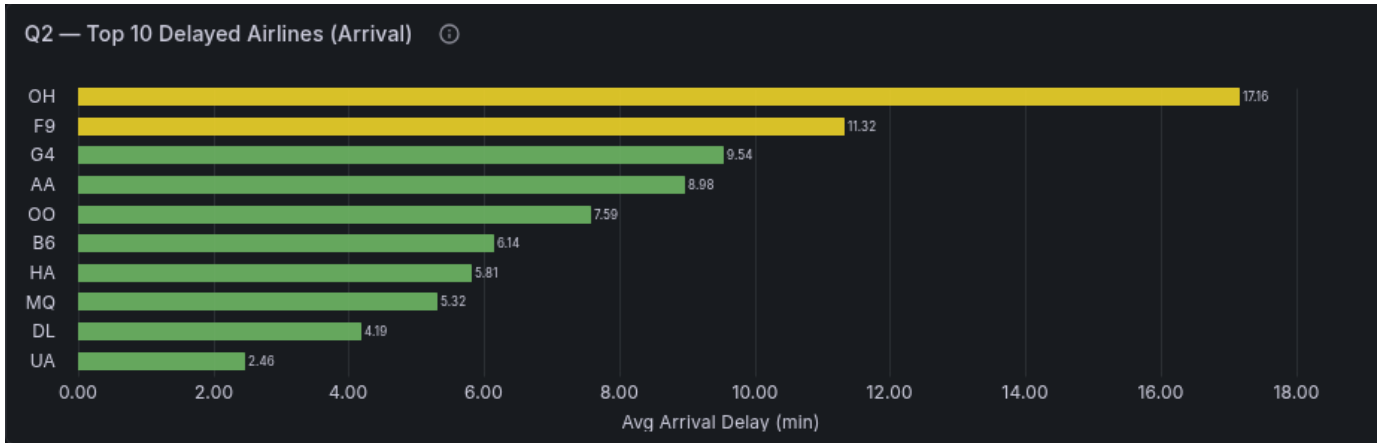


Figura 13. Risultati Query 2: Classifica delle 10 tratte con i ritardi medi più elevati.

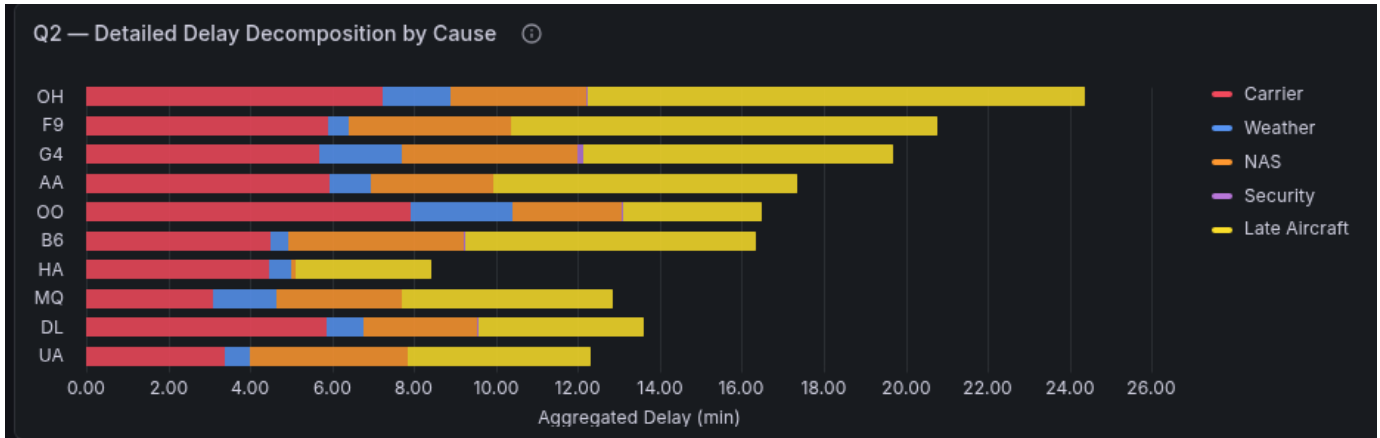


Figura 14. Risultati Query 2: Decomposizione percentuale delle cause di ritardo.

C. Query 3

Per la Query 3, vengono presentati, per le quattro compagnie aeree in esame, la distribuzione mediana oraria e il trend dei percentili critici stimati.

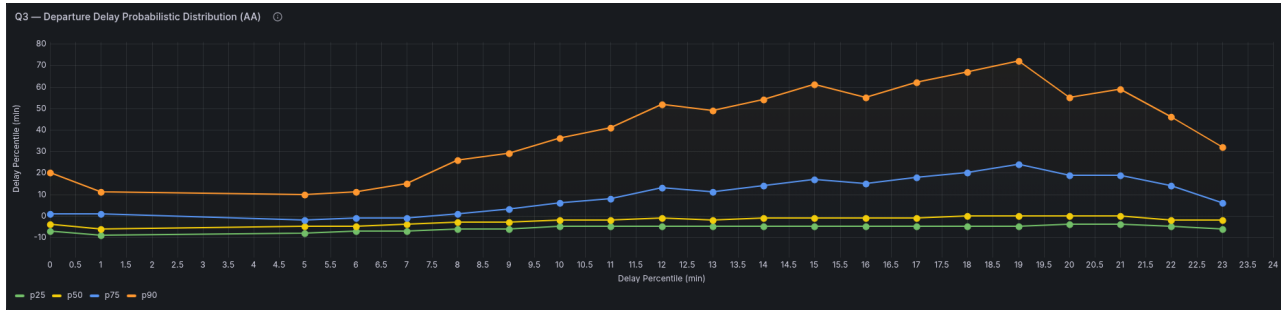


Figura 15. Risultati Query 3 (AA): Trend orario dei percentili di ritardo critici.

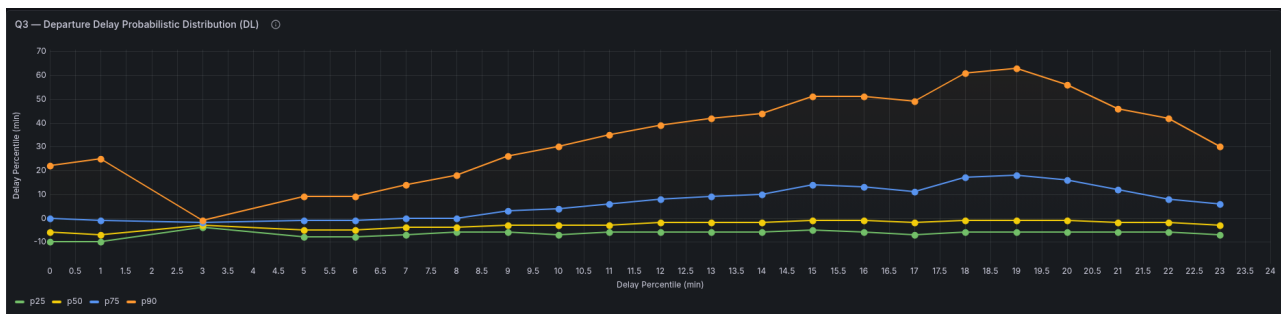


Figura 16. Risultati Query 3 (DL): Trend orario dei percentili di ritardo critici.

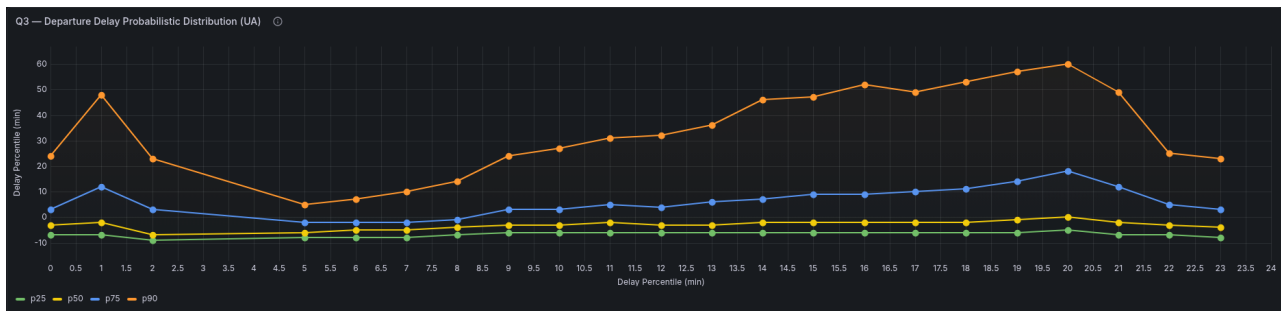


Figura 17. Risultati Query 3 (UA): Trend orario dei percentili di ritardo critici.

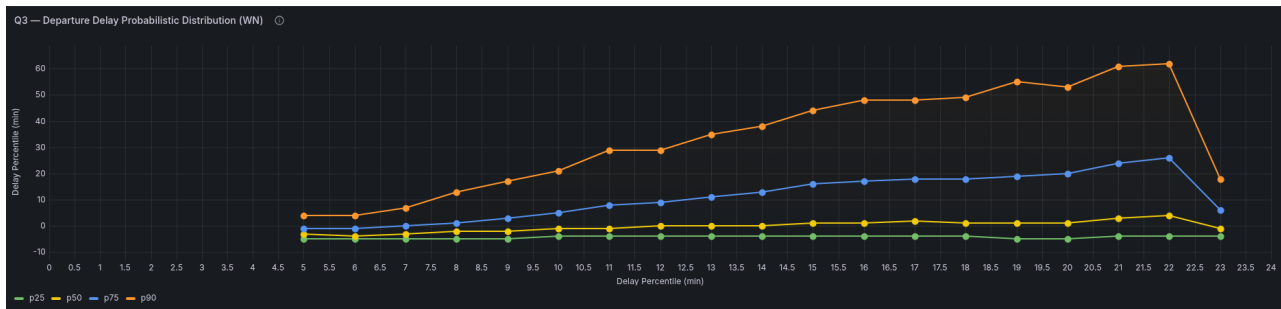


Figura 18. Risultati Query 3 (WN): Trend orario dei percentili di ritardo critici.

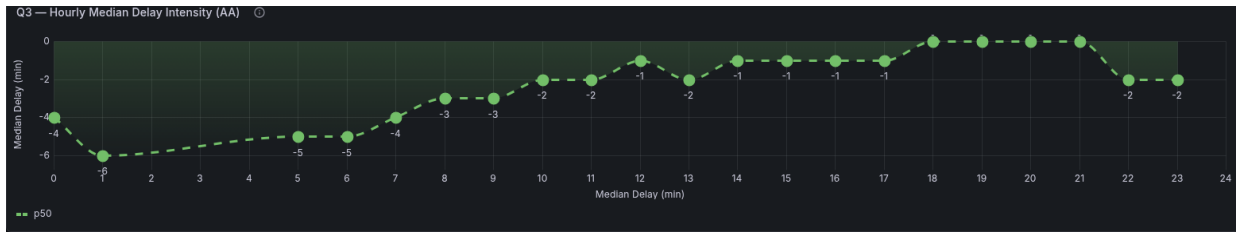


Figura 19. Risultati Query 3 (AA): Distribuzione oraria del ritardo medio.

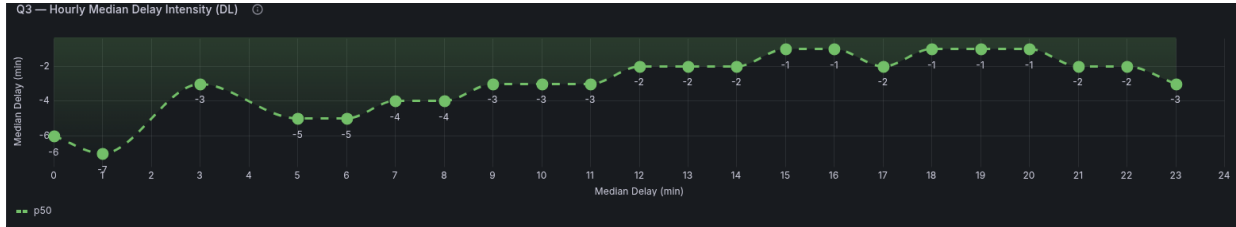


Figura 20. Risultati Query 3 (DL): Distribuzione oraria del ritardo medio.

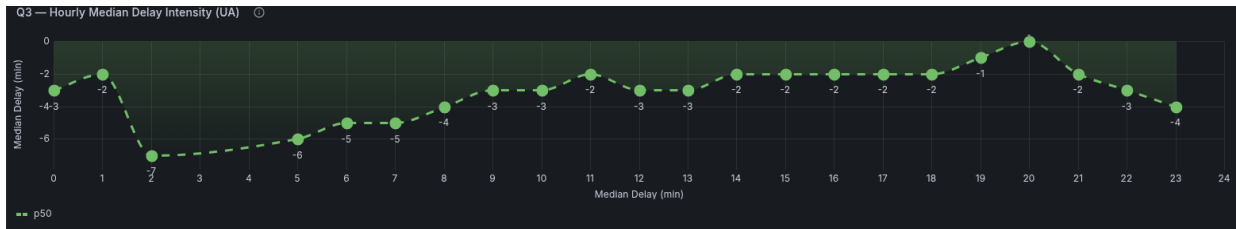


Figura 21. Risultati Query 3 (UA): Distribuzione oraria del ritardo medio.



Figura 22. Risultati Query 3 (WN): Distribuzione oraria del ritardo medio.



Figura 23. Risultati Query 3: Valori estremi dei ritardi per American Airlines (AA), Delta Air Lines (DL), United Airlines (UA) e Southwest Airlines (WN) (da sinistra a destra).

D. Query 4

Per la Query 4, vengono presentati i risultati relativi alla distribuzione geometrica dei cluster identificati tramite PCA e le coordinate dei centroidi per ciascun cluster.

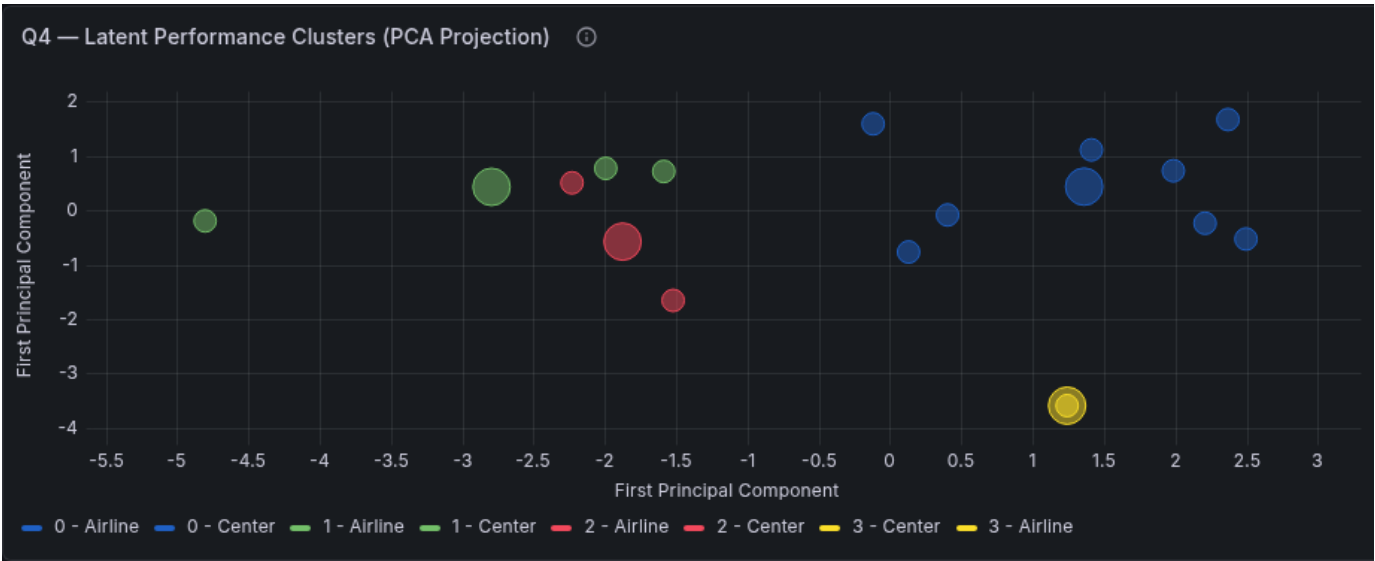


Figura 24. Risultati Query 4: Distribuzione geometrica dei cluster estratti tramite PCA.

Tabella I
RISULTATI QUERY 4: METRICHE OPERATIVE MEDIE (VALORI GREZZI) E CLASSIFICAZIONE.

ID	Dep.	Arr.	Makeup	StdDev	Div.%	Can.%	Clus.	Carr.	Weat.	NAS	Secu.	Late Air.
UA	9.03	2.46	-6.52	47.56	0.21	0.69	0	18.85	3.40	21.33	0.00	24.93
NK	9.17	2.39	-6.77	48.28	0.13	1.41	0	17.50	1.59	34.29	0.19	16.98
B6	12.51	6.14	-6.33	56.11	0.25	1.23	0	19.44	1.88	18.64	0.11	30.64
DL	10.71	4.19	-6.40	55.11	0.19	0.94	0	31.19	4.75	14.85	0.04	21.56
MQ	8.44	5.32	-3.12	50.71	0.20	2.82	0	15.52	7.75	15.35	0.09	25.84
YX	4.78	0.69	-4.04	42.22	0.16	2.26	0	16.85	3.15	21.93	0.04	20.26
AS	5.52	1.30	-4.22	36.08	0.16	1.00	0	14.58	2.35	14.53	0.22	19.04
WN	9.35	1.76	-7.58	35.25	0.16	1.14	0	15.26	1.61	11.46	0.14	28.27
AA	14.59	8.98	-5.59	74.97	0.25	1.94	1	27.11	4.61	13.51	0.11	33.72
F9	16.00	11.32	-4.62	71.65	0.16	1.67	1	23.05	1.91	15.36	0.00	40.53
OH	19.78	17.16	-2.60	82.75	0.21	6.09	1	25.98	5.96	11.86	0.12	43.51
OO	11.68	7.59	-4.04	62.92	0.24	1.47	2	38.93	12.11	13.15	0.10	16.58
G4	13.87	9.54	-4.31	78.39	0.23	0.66	2	24.89	8.93	18.70	0.68	33.02
HA	6.35	5.81	-0.52	37.42	0.10	1.21	3	27.16	3.28	0.58	0.15	19.99

Legenda:

Dep./Arr.: ritardo medio partenza/arrivo;
Makeup:capacità di recupero ritardo in volo;
StdDev: deviazione standard ritardo arrivo;
Div.%/Can.%: tasso deviazioni e cancellazioni;
Clus.: ID Cluster;
Carr./Weat./NAS/Secu./Late Air.: cause di ritardo medie in minuti;